Алгоритм Кнута-Морриса-Пратта

Стариков Егор, 25.Б42

История и решаемая задача

- Алгоритм был открыт Д. Кнутом и В. Праттом и, независимо от них, Д. Моррисом. Результаты своей работы они совместно опубликовали в 1977 году.
- Решает задачу поиска подстроки в строке
  - Краулеры
  - Ctrl+F на странице
  - Анализ ДНК
  - Поиск сигнатур вирусов в коде

Префикс-функция

- Определение

$$\pi[i] = \max_{1 \le k \le i} \{k : S[0 : k-1] = S[i-k+1 : i]\}$$

Пример

- 1)

$a\,b\,a\,b\,b\,a\,b\,a$ $\quad$ $[0\,0\;\;\;\;\;\;\;\;\;\;]$

$\bar{a}\,b\,\bar{a}\,b\,b\,a\,b\,a$ $\quad$ $[0\,0\,1\;\;\;\;\;\;\;\;]$

$\overline{a\,b}\,\overline{a\,b}\,b\,a\,b\,a$ $\quad$ $[0\,0\,1\,2\;\;\;\;\;\;]$

$a\,b\,a\,b\,b\,a\,b\,a$ $\quad$ $[0\,0\,1\,2\,0\;\;\;\;]$

$\bar{a}\,b\,a\,b\,b\,\bar{a}\,b\,a$ $\quad$ $[0\,0\,1\,2\,0\,1\;\;\;]$

$\overline{a\,b}\,a\,b\,b\,\overline{a\,b}\,a$ $\quad$ $[0\,0\,1\,2\,0\,1\,2\;]$

$\overline{a\,b\,a}\,b\,b\,\overline{a\,b\,a}$ $\quad$ $[0\,0\,1\,2\,0\,1\,2\,3]$

- 2)

$[a\,a\,a\,b\,a\,a\,a\,b\,a\,a\,a\,a]$
$[0\,1\,2\,0\,1\,2\,3\,4\,5\,6\,7\;\;]$

$[a\,a\,a\,b\,a\,a\,a\,b\,a\,a\,a\,a]$
$[0\,1\,2\,0\,1\,2\,3\,4\,5\,6\,7\;\;]$

$[a\,a\,a\,b\,a\,a\,a\,b\,a\,a\,a\,a]$
$[0\,1\,2\,0\,1\,2\,3\,4\,5\,6\,7\;\;]$

$[a\,a\,a\,b\,a\,a\,a\,b\,a\,a\,a\,a]$
$[0\,1\,2\,0\,1\,2\,3\,4\,5\,6\,7\,3]$

Префикс-функция

```
void prefixFunc(const char* s, int* pi, size_t n)
{
    pi[0] = 0;

    for (size_t i = 1; i < n; i++) {
        int j = pi[i - 1];
        while ((j > 0) && (s[i] != s[j])) {
            j = pi[j - 1];
        }
        if (s[i] == s[j]) {
            j++;
        }
        pi[i] = j;
    }
}
```

- Количество увеличений <= n за всю программу. Следовательно, количество уменьшений в while <= n за всю программу.

$$N_{\text{уменьш}} = J_0 - J_{final} + N_{\text{увел}}$$

$$J_0 = 0;\ J_{final} \geq 0;\ N_{\text{увел}} \leq n$$

Алгоритм КМП

```c
void kmpSearch(char* text, char* pattern)
{
    size_t n = strlen(text);
    size_t m = strlen(pattern);

    int* pi = (int*)malloc(m * sizeof(int));
    if (!pi) {
        return;
    }

    prefixFunc(pattern, pi, m);

    int j = 0;

    for (size_t i = 0; i < n; i++) {
        while (j > 0 && text[i] != pattern[j])
            j = pi[j - 1];

        if (text[i] == pattern[j])
            j++;

        if (j == m) {
            printf("  Найдено в позиции %zu (индекс с 0)\n", i - m + 1);
            j = pi[j - 1];
        }
    }

    free(pi);
}
```

- Строится префикс-функция для искомой подстроки, далее алгоритм делает то же самое, что и обычная префикс-функция, но не добавляет в массив, храня только последнее значение.

Спасибо за внимание